

4.13.79

Dhanush Kumar A - AI25BTECH11010

October 3, 2025

Question

Solve the following equations for x and y using cross-multiplication method

$$(ax - by) + (a + 4b) = 0$$

$$(bx + ay) + (b - 4a) = 0$$

Solution

The system of equations can be written in vector-matrix form as:

$$\begin{pmatrix} a \\ -b \end{pmatrix}^T \mathbf{X} = -a - 4b \quad (1)$$

$$\begin{pmatrix} b \\ a \end{pmatrix}^T \mathbf{X} = 4a - b \quad (2)$$

Using Cramer's Rule (matrix method), which is algebraically equal to the cross-multiplication method, the solution is obtained as follows.

The coefficient matrix and its determinant are:

$$A = \begin{pmatrix} a & -b \\ b & a \end{pmatrix} \quad (3)$$

$$\det(A) = a^2 + b^2 \quad (4)$$

Solution

The determinant for the first component of $\mathbf{X}$ is obtained by replacing the first column with the constants:

$$A_x = \begin{pmatrix} -a - 4b & -b \\ 4a - b & a \end{pmatrix} \quad (5)$$

$$\det(A_x) = (-a - 4b)a - (-b)(4a - b) \quad (6)$$

$$= -a^2 - 4ab + 4ab - b^2 \quad (7)$$

$$= -(a^2 + b^2) \quad (8)$$

The determinant for the second component of $\mathbf{X}$ is obtained by replacing the second column with the constants:

$$A_y = \begin{pmatrix} a & -a - 4b \\ b & 4a - b \end{pmatrix} \quad (9)$$

$$\det(A_y) = a(4a - b) - b(-a - 4b) \quad (10)$$

$$= 4a^2 - ab + ab + 4b^2 \quad (11)$$

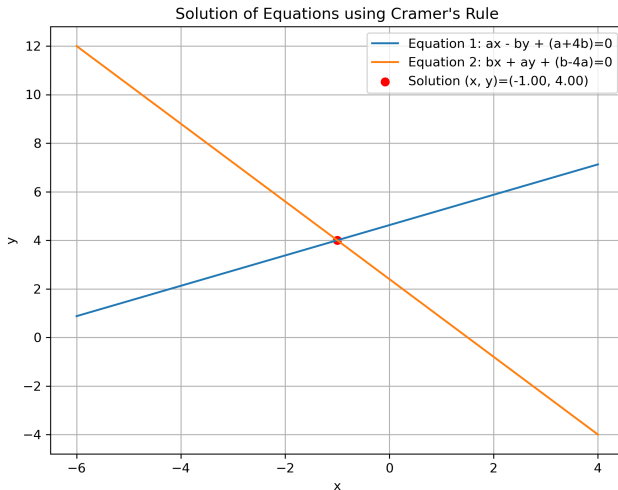
$$= 4(a^2 + b^2) \quad (12)$$

Solution

Finally, using Cramer's Rule, the solution vector is:

$$\mathbf{x} = \frac{1}{\det(A)} \begin{pmatrix} \det(A_x) \\ \det(A_y) \end{pmatrix} \quad (13)$$

$$\mathbf{x} = \begin{pmatrix} -1 \\ 4 \end{pmatrix} \quad (14)$$



Python code - To solve and plot

```
import numpy as np
import matplotlib.pyplot as plt
import os

# Create folder 'figs' if it doesn't exist
if not os.path.exists("figs"):
    os.makedirs("figs")

# Coefficients
a = float(input("Enter value of a: "))
b = float(input("Enter value of b: "))

# Coefficient matrix A
A = np.array([[a, -b],
              [b, a]])

# Constants vector
B = np.array([- (a + 4*b),
              4*a - b])
```

Python code - To solve and plot

```
# Determinants
det_A = np.linalg.det(A)
A_x = np.array([[B[0], -b],
                [B[1], a]])
det_Ax = np.linalg.det(A_x)

A_y = np.array([[a, B[0]],
                [b, B[1]]])
det_Ay = np.linalg.det(A_y)

# Solution
x = det_Ax / det_A
y = det_Ay / det_A

print(f"Solution:\nx = {x}\ny = {y}")
```


Python code - To solve and plot

```
# Plotting
x_vals = np.linspace(x-5, x+5, 400)

# Equation 1: (ax - by) + (a+4b)=0 -> y = (ax + a+4b)/b
y1 = (a*x_vals + (a + 4*b)) / b

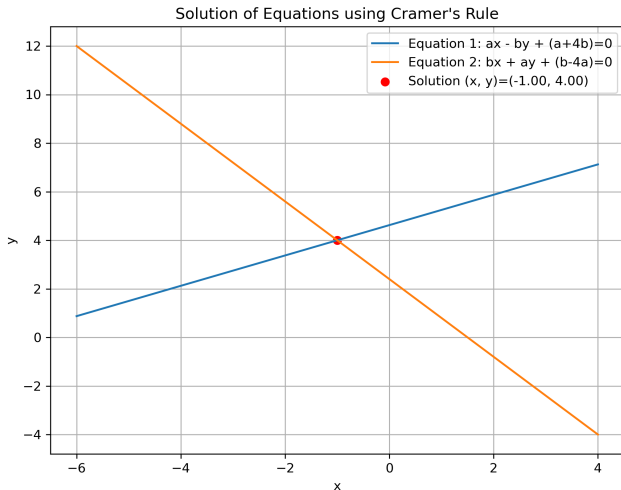
# Equation 2: (bx + ay) + (b-4a)=0 -> y = -(b*x_vals + b-4a)/a
y2 = -(b*x_vals + (b - 4*a)) / a
```

Python code - To solve and plot

```
plt.figure(figsize=(8,6))
plt.plot(x_vals, y1, label='Equation 1:  $ax - by + (a+4b)=0$ ')
plt.plot(x_vals, y2, label='Equation 2:  $bx + ay + (b-4a)=0$ ')
plt.scatter(x, y, color='red', label=f'Solution (x, y)=({x:.2f}, {y:.2f})')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Solution of Equations using Cramer\'s Rule')
plt.legend()
plt.grid(True)

# Save figure
plt.savefig("../figs/equations_solution.png", dpi=300)
plt.show()
```

Plot-Using Python



C code - Solving

```
#include <stdio.h>
#include <stdlib.h>

// Function to solve 2x2 linear system using Cramer's rule
// Equations:
// (a*x - b*y) + (a+4b) = 0
// (b*x + a*y) + (b-4a) = 0
// Inputs: a, b
// Outputs: x, y (via pointers)
void solve_2x2(double a, double b, double *x, double *y) {
    // Coefficient matrix
    double det = a*a + b*b; // det = a*a + b*b

    if(det == 0) {
        // Singular system, return NaN
        *x = 0.0/0.0;
        *y = 0.0/0.0;
        return;
    }
}
```

C code - Solving

```
// Determinants for x and y
double det_x = -((a + 4*b)*a + (b - 4*a)*b);
double det_y = -((a + 4*b)*b - (b - 4*a)*a);

*x = det_x / det;
*y = det_y / det;
}
```

Python code -Ploting the plane using c function

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt
import os

# Ensure 'figs' folder exists
os.makedirs("figs", exist_ok=True)

# Load the shared library
lib = ctypes.CDLL("./main.so")

# Define argument types for ctypes
lib.solve_2x2.argtypes = [
    ctypes.c_double, ctypes.c_double,
    ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.c_double)
]
```

Python code -Ploting the plane using c function

```
# Read input coefficients
a = float(input("Enter value of a: "))
b = float(input("Enter value of b: "))

# Prepare output variables
x = ctypes.c_double()
y = ctypes.c_double()

# Call C function
lib.solve_2x2(a, b, ctypes.byref(x), ctypes.byref(y))

# Extract values
x_val = x.value
y_val = y.value

print(f"Solution from C function:\nx = {x_val}\ny = {y_val}")
```

Python code -Plotting the plane using c function

```
# ----- Plotting -----  
  
# Generate x values around the solution  
x_vals = np.linspace(x_val-5, x_val+5, 400)  
  
# Equation 1:  $(a*x - b*y) + (a+4b) = 0 \rightarrow y = (a*x + (a+4b))/b$   
y1 = (a*x_vals + (a + 4*b)) / b  
  
# Equation 2:  $(b*x + a*y) + (b-4a) = 0 \rightarrow y = -(b*x + (b-4a))/a$   
y2 = -(b*x_vals + (b - 4*a)) / a
```


Python code -Ploting the plane using c function

```
plt.figure(figsize=(8,6))
plt.plot(x_vals, y1, label=f'Equation 1: {a}x - {b}y + ({a+4*b})
        =0')
plt.plot(x_vals, y2, label=f'Equation 2: {b}x + {a}y + ({b-4*a})
        =0')
plt.scatter(x_val, y_val, color='red', s=100, label=f'Solution ({
        x_val:.2f}, {y_val:.2f})')
plt.xlabel('x')
plt.ylabel('y')
plt.title("Solution of 2x2 Linear System using C Function")
plt.legend()
plt.grid(True)

# Save figure
fig_path = os.path.join("../figs", "equations_solution1.png")
plt.savefig(fig_path, dpi=300)
print(f"Figure saved at: {fig_path}")
```

Plot-Using Python and C

